

Documentação técnica do projeto Challenger 2 – Fiap

Grupo:

Azevedo Renan

Humberto Mauro

Joao Paulo

Arquitetura:

Não adotamos arquitetura alguma e sim o padrão de projeto (design) “MVC”. Como a implementação do MVC é tão central que a arquitetura da aplicação é comumente referida, de forma simplificada, como uma arquitetura “baseada em MVC” com classes concretas.

Requisitos técnicos:

Back-end em Node.js, v22.19.0 (mais recente, pelo menos até a escrita desse documento)

Utilização de frameworks Express para roteamento e sequelize (ORM). Migration para criação e alteração de tabelas do banco.

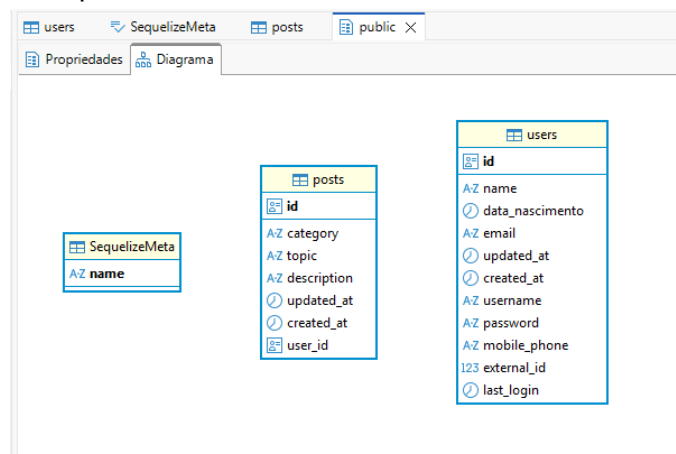
Persistência de dados: Banco de dados PostgreSQL.

Tabelas do Postgres: criadas e alteradas via ORM sequelize.

Duas tabelas cradas: posts e users, sem relacionamento de chave estrangeira .

Controle será da aplicação.

A sequelizeMeta é do ORM.



Containerização com Docker: Desenvolvimento e implantação usando contêineres Docker para garantir consistência entre ambientes de desenvolvimento e produção.

Linux: ubuntu latest

Testes unitários: uso do “Axios” para testes.

Foi utilizado o Javascript, nativo do nodes.js

Middleware: Não utilizamos nenhum token.

Setup inicial:

npm init -y

npm install express pg

npm install --save-dev sequelize-cli

npm install axios --save-dev

Execução:

app:

Modo dev: node --watch src/server.js

Modo prod: node src/server.js

sequelize-cli:

criar migration:

npx sequelize-cli migration:generate --name create_users_table

subir (up) migration:

npx sequelize-cli db:migrate

testes:

node src/tests/userTests.js

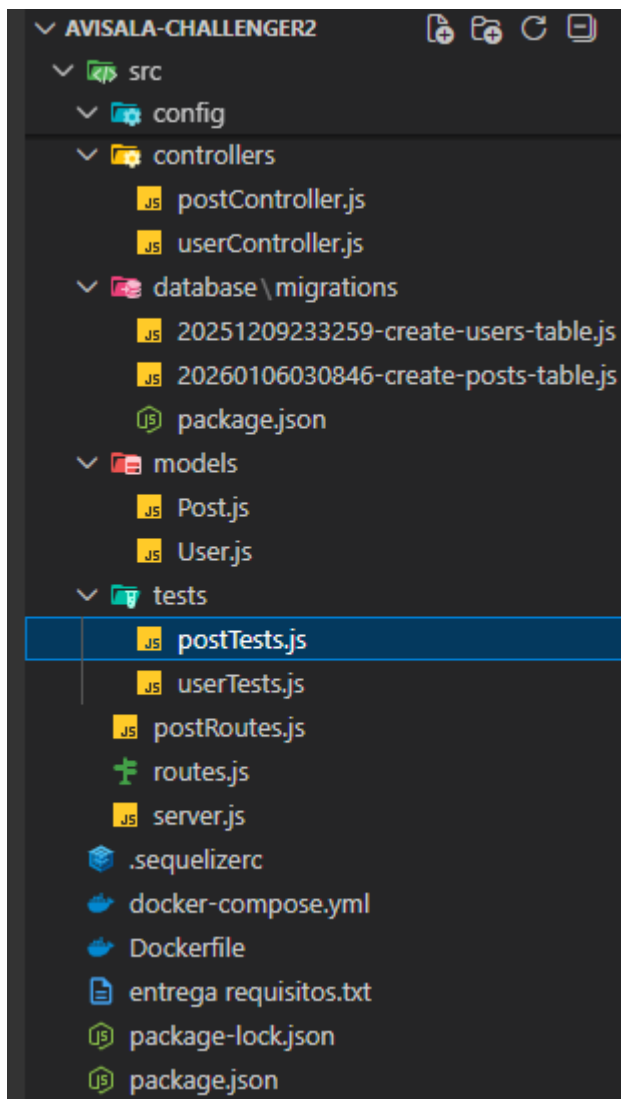
node src/tests/postTests.js

Editor: VScode e como plug-in o Thunder Client para requisições Restfull

Docker: workflow, Dockerfile e compose.

docker compose up --build

Estrutura:



A aplicação se inicia no /src/server.js

Quando um verbo é chamado na URL, a API identifica qual rota chamar pelo que está no server

```
app.use('/users', userRoutes)
app.use('/posts', postRoutes)
```

Direciona para a rota de acordo com que está na url

```

1 import express from 'express'
2 import { createPost, getAllPosts, deletePost, getPostByID, updatePost, searchDescriptionByKeyword, searchByCategory } from './controllers'
3
4 const router = express.Router()
5
6 router.post('/', createPost)
7 router.put('/:id', updatePost);
8 router.delete('/:id', deletePost)
9 router.get('/', getAllPosts)
10 router.get('/:id', getPostByID);
11
12 router.get('/search', searchDescriptionByKeyword)
13 router.get('/category/:category', searchByCategory);
14
15 //Posts
16
17 export default router

```

e executa o método da Controller, de acordo com o parâmetro da rota.

Então temos:

URL://localhost:3000/users

Para users:

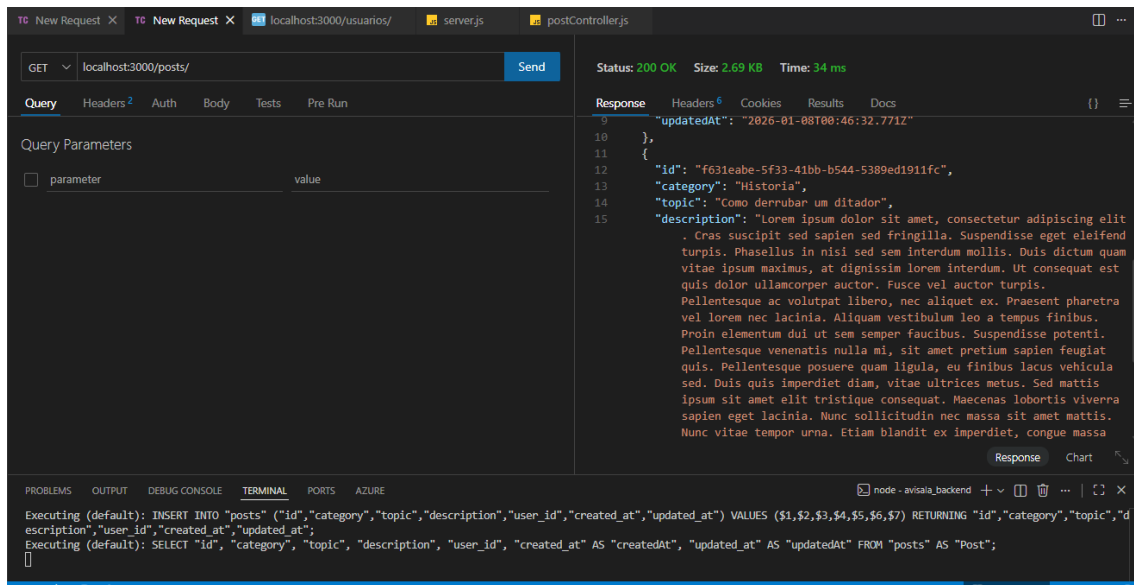
VERBOS	ROTA	AÇÃO
POST	/	createUser
GET	/	getAllUsers
DELETE	/:id	deleteUser
PUT	/:id	updateUser
GET	/:id	getUserByID
GET	/search	searchByPalavraChave
GET	/loginUsuario	loginUsuario

Para Posts:

URL://localhost:3000/posts

VERBOS	ROTA	AÇÃO
POST	/	createPost
GET	/	getAllPosts
DELETE	/:id	deletePost
PUT	/:id	updatePost
GET	/:id	getPostByID
GET	/search	searchDescriptionByKeyword
GET	/category/:category	searchByCategory

Estando o container do postgres sendo executado e iniciando o app:



Retorno os 2 posts:

"id": "51349ce6-85de-49f7-8eb2-539d66a46f12", "category": "Matematica",

"topic": "Como calcular o perímetro do círculo" ...demais dados e

"id": "f631eabe-5f33-41bb-b544-5389ed1911fc", "category": "Historia", "topic":

"Como derrubar um ditador",.. demais dados.

id	category	topic	description	updated_at	created_at
1	Matematica	Como calcular o perímetro do círculo	Lorem ipsum dolor sit amet, consectetur adipiscing elit.	2026-01-07 21:46:32.771 -0300	2026-01-07 21:46:32.771 -0300
2	Historia	Como derrubar um ditador	Lorem ipsum dolor sit amet, consectetur adipiscing elit.	2026-01-07 21:58:05.282 -0300	2026-01-07 21:58:05.282 -0300

PUT	/:id	updatePost
-----	------	------------

localhost:3000/posts/f631eabe-5f33-41bb-b544-5389ed1911fc

body:

{

"category": "Historia",

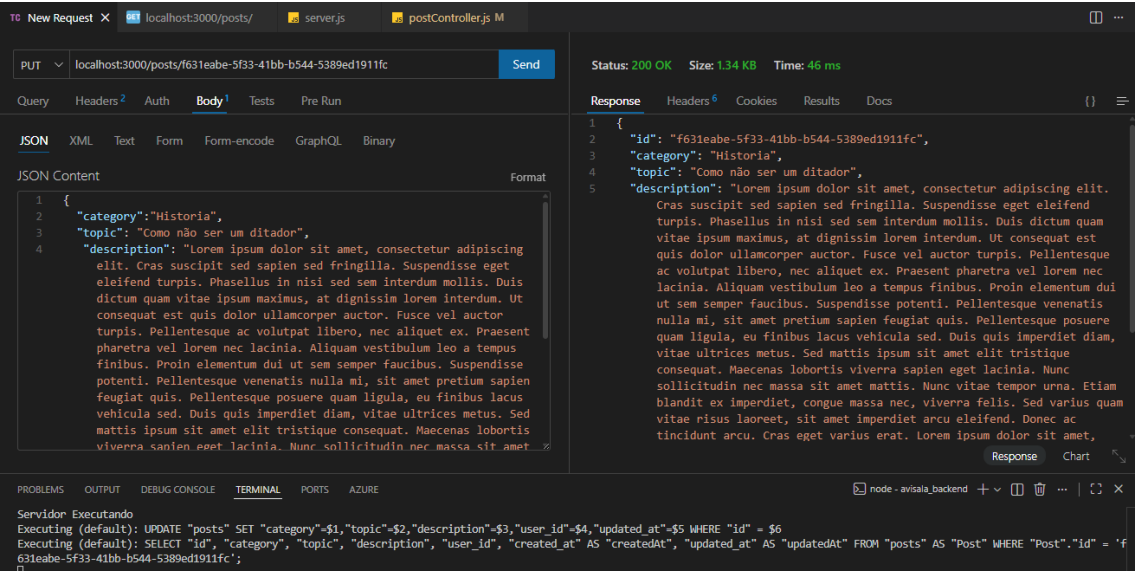
"topic": "Como não ser um ditador",

"description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est quis dolor ullamcorper auctor. Fusce vel auctor turpis. Pellentesque ac volutpat libero, nec aliquet ex. Praesent pharetra vel lorem nec lacinia. Aliquam vestibulum leo a tempus finibus. Proin elementum dui ut sem semper faucibus. Suspendisse potenti. Pellentesque venenatis nulla mi, sit amet pretium sapien feugiat quis. Pellentesque posuere quam ligula, eu finibus lacus vehicula sed. Duis quis imperdiet diam, vitae ultrices metus. Sed mattis ipsum sit

amet elit tristique consequat. Maecenas lobortis viverra sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis. Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse a dolor velit.",

"user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509"

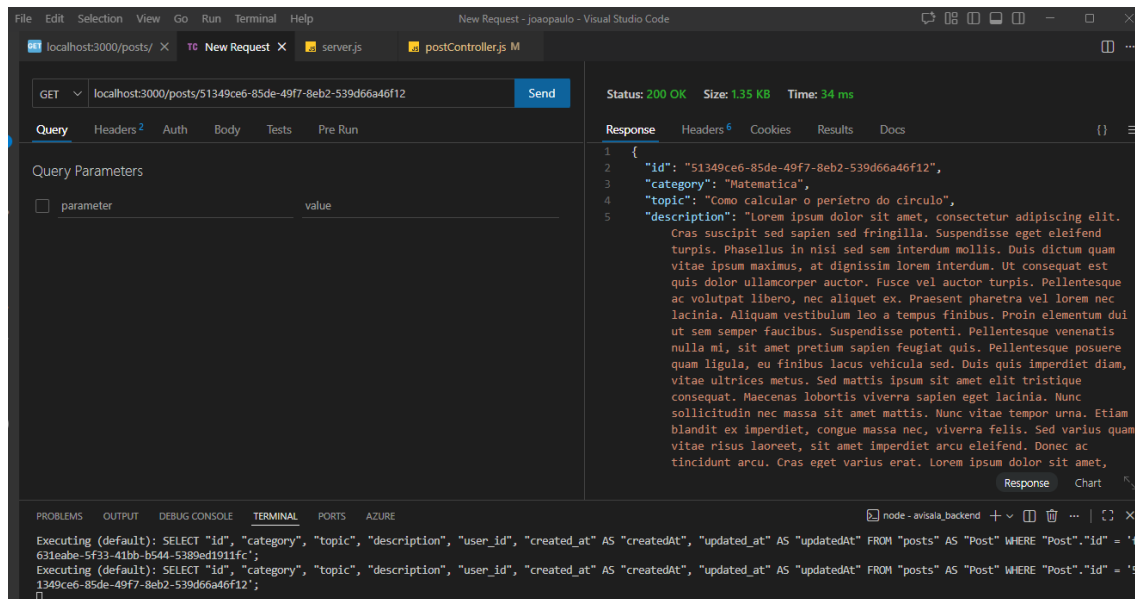
}



id	category	topic	description	updated_at	created_at
1	Matematica	Como calcular o perimetro do círculo	Lorem ipsum dolor sit amet, consectetur adipiscing elit.	2026-01-07 21:46:32.771 -0300	2026-01-07 21:46:32.771 -0300
2	Historia	Como não ser um ditador	Lorem ipsum dolor sit amet, consectetur adipiscing elit.	2026-01-07 22:26:24.053 -0300	2026-01-07 21:58:05.28 -0300

GET	/:id	getPostById
-----	------	-------------

localhost:3000/posts/51349ce6-85de-49f7-8eb2-539d66a46f12



E pode ser também, passando o param “search”, assim passa a atuar como “searchDescriptionByKeyword”

localhost:3000/posts/Search

body

```
{
  "topic": "perímetro"
}
```

A pesquisa pode ser feita pela description ou pelo topic. No caso foi usado o topic.

```
export const searchDescriptionByKeyword = async (req, res) => {
  const posts = await Post.findAll({
    where: {
      [Op.or]: {
        description: {[Op.like]: "%" + req.body.description + "%"},
        topic: {[Op.like]: "%" + req.body.topic + "%"}
      }
    }
  });
  res.status(200).json(posts)
}
```


localhost:3000/posts/ TO New Request X server.js postController.js M

GET localhost:3000/posts/search Send

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "topic": "perimetro"
3 }
```

Status: 200 OK Size: 1.35 KB Time: 6 ms

Response Headers Cookies Results Docs

```
1 [
2   {
3     "id": "51349ce6-85de-49f7-8eb2-539d66a46f12",
4     "category": "Historia",
5     "topic": "Como calcular o perimetro do circulo",
6     "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit
. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend
turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam
vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est
quis dolor ullamcorper auctor. Fusce vel auctor turpis.
Pellentesque ac volutpat libero, nec aliquet ex. Praesent pharetra
vel lorem nec lacinia. Aliquam vestibulum leo a tempus finibus.
Proin elementum dui ut semper faucibus. Suspendisse potenti.
Pellentesque venenatis nulla mi, sit amet pretium sapien feugiat
quis. Pellentesque posuere quam ligula, eu finibus lacus vehicula
sed. Duis quis imperdiet diam, vitae ultrices metus. Sed mattis
ipsum sit amet elit tristique consequat. Maecenas lobortis viverra
sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis.
Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa
nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

Restarting 'src/server.js'
Executing (default): SELECT 1+1 AS result
BD conectado
Servidor Executando

localhost:3000/posts/ TO New Request X server.js postController.js M

GET localhost:3000/posts/search Send

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "description": "ipsum"
3 }
```

Status: 200 OK Size: 2.69 KB Time: 61 ms

Response Headers Cookies Results Docs

```
1 ipsum sit amet elit tristique consequat. Maecenas lobortis viverra
2 sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis.
3 Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa
4 nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet
5 imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius
6 erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
7 Suspendisse a dolor velit.",
8 "user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509",
9 "createdAt": "2026-01-08T00:58:05.282Z",
10 "updatedAt": "2026-01-08T01:26:24.053Z"
11 },
12 {
13   "id": "51349ce6-85de-49f7-8eb2-539d66a46f12",
14   "category": "Historia",
15   "topic": "Como calcular o perimetro do circulo",
16   "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit
. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend
turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam
vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est
quis dolor ullamcorper auctor. Fusce vel auctor turpis.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

BD conectado
Servidor Executando
Executing (default): SELECT "id", "category", "topic", "description", "user_id", "created_at" AS "createdAt", "updated_at" AS "updatedAt" FROM "posts" AS "Post" WHERE ("Post"."description" LIKE '%ipsum%' OR "Post"."topic" LIKE '%undefined%');

localhost:3000/posts/ TO New Request X server.js postController.js M postRoutes.js M

GET localhost:3000/posts/search Send

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "description": "ipsum",
3   "topic": "perimetro"
4 }
```

Status: 200 OK Size: 2.69 KB Time: 31 ms

Response Headers Cookies Results Docs

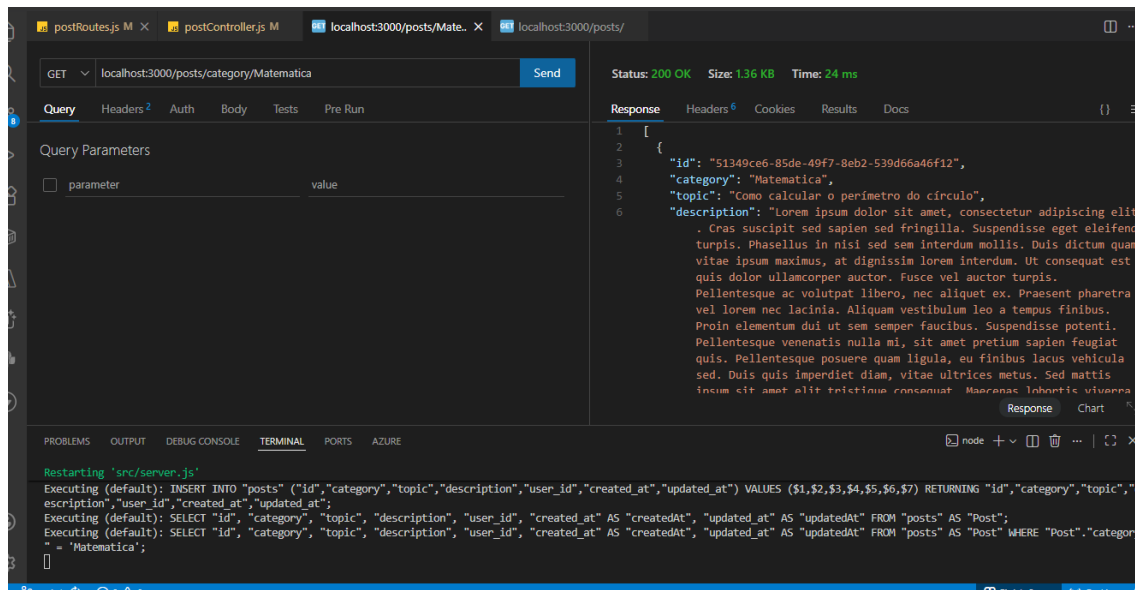
```
1 ipsum sit amet elit tristique consequat. Maecenas lobortis viverra
2 sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis.
3 Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa
4 nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet
5 imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius
6 erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
7 Suspendisse a dolor velit.",
8 "user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509",
9 "createdAt": "2026-01-08T00:58:05.282Z",
10 "updatedAt": "2026-01-08T01:26:24.053Z"
11 },
12 {
13   "id": "51349ce6-85de-49f7-8eb2-539d66a46f12",
14   "category": "Historia",
15   "topic": "Como calcular o perimetro do circulo",
16   "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit
. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend
turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam
vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est
quis dolor ullamcorper auctor. Fusce vel auctor turpis.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

BD conectado
Servidor Executando
Executing (default): SELECT "id", "category", "topic", "description", "user_id", "created_at" AS "createdAt", "updated_at" AS "updatedAt" FROM "posts" AS "Post" WHERE ("Post"."description" LIKE '%ipsum%' OR "Post"."topic" LIKE '%perimetro%');

GET	/:category	searchByCategory
-----	------------	------------------

localhost:3000/posts/Matematica

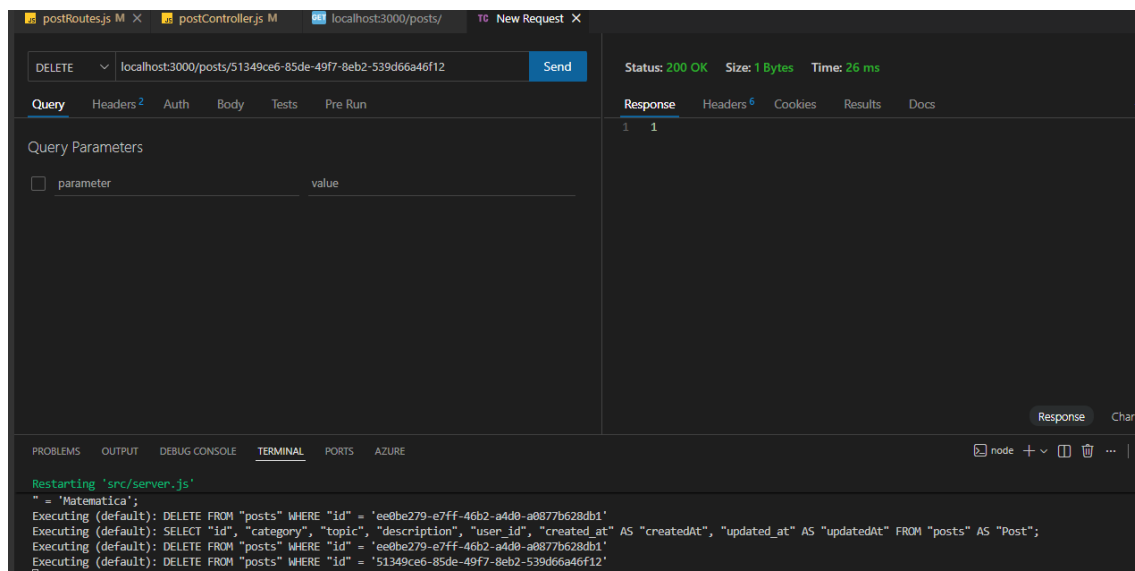


S\u00f3 h\u00e1 1 registro com categoria Matematica

DELETE	/:id	deletePost
--------	------	------------

localhost:3000/posts/51349ce6-85de-49f7-8eb2-539d66a46f12

Executing (default): DELETE FROM "posts" WHERE "id" = '51349ce6-85de-49f7-8eb2-539d66a46f12'



E por fim, listando todos os posts

postRoutes.js M × postController.js M localhost:3000/posts/ × New Request

GET localhost:3000/posts/ Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content 1 Format

Status: 200 OK Size: 1.34 KB Time: 29 ms

Response Headers 6 Cookies Results Docs {}

4 "category": "Historia",
5 "topic": "Como não ser um ditador",
6 "description": "Lorem ipsum dolor sit amet, consectetur adipiscing elit
. Cras suscipit sed sapien sed fringilla. Suspendisse eget eleifend
turpis. Phasellus in nisi sed sem interdum mollis. Duis dictum quam
vitae ipsum maximus, at dignissim lorem interdum. Ut consequat est
quis dolor ullamcorper auctor. Fusce vel auctor turpis.
Pellentesque ac volutpat libero, nec aliquet ex. Praesent pharetra
vel lorem nec lacinia. Aliquam vestibulum leo a tempus finibus.
Proin elementum dui ut semper faucibus. Suspendisse potenti.
Pellentesque venenatis nulla mi, sit amet pretium sapien feugiat
quis. Pellentesque posuere quam ligula, eu finibus lacus vehicula
sed. Duis quis imperdiet diam, vitae ultrices metus. Sed mattis
ipsum sit amet elit tristique consequat. Maecenas lobortis viverra
sapien eget lacinia. Nunc sollicitudin nec massa sit amet mattis.
Nunc vitae tempor urna. Etiam blandit ex imperdiet, congue massa
nec, viverra felis. Sed varius quam vitae risus laoreet, sit amet
imperdiet arcu eleifend. Donec ac tincidunt arcu. Cras eget varius
erat. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Suspendisse a dolor velit.",
7 "user_id": "3c3ff0c9-e8d8-441e-ac00-3bbc2ab2a509",
8 "createdAt": "2026-01-08T00:58:05.282Z",
9 "updatedAt": "2026-01-08T00:56:34.073Z"

Response Chart

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

Executing (default): SELECT "id", "category", "topic", "description", "user_id", "created_at" AS "createdAt", "updated_at" AS "updatedAt" FROM "posts" AS "Post";

POST	/	createUser
------	---	------------

url: localhost:3000/users/

POST localhost:3000/users/ Send

Status: 201 Created Size: 306 Bytes Time: 119 ms

Response

```
1 {
2   "id": "85875e79-7077-4861-972b-9be4ef59fc73",
3   "name": "fulano",
4   "data_nascimento": "2001-01-01",
5   "email": "fulanoo@gmail.com",
6   "username": "fulano80",
7   "password": "1234as",
8   "mobile_phone": "11999999999",
9   "updatedAt": "2026-01-18T16:23:05.271Z",
10  "createdAt": "2026-01-18T16:23:05.271Z",
11  "external_id": null,
12  "last_login": null
13 }
```

DBeaver 25.3.1 - users

Arquivo Editar Navegar Procurar Editor SQL Banco de dados Janela Ajuda

SQL Commit Rollback Auto postgres public@avisdadb

users SequelizeMeta posts

Filtra conexões por nome

Show SQL Inserir uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)

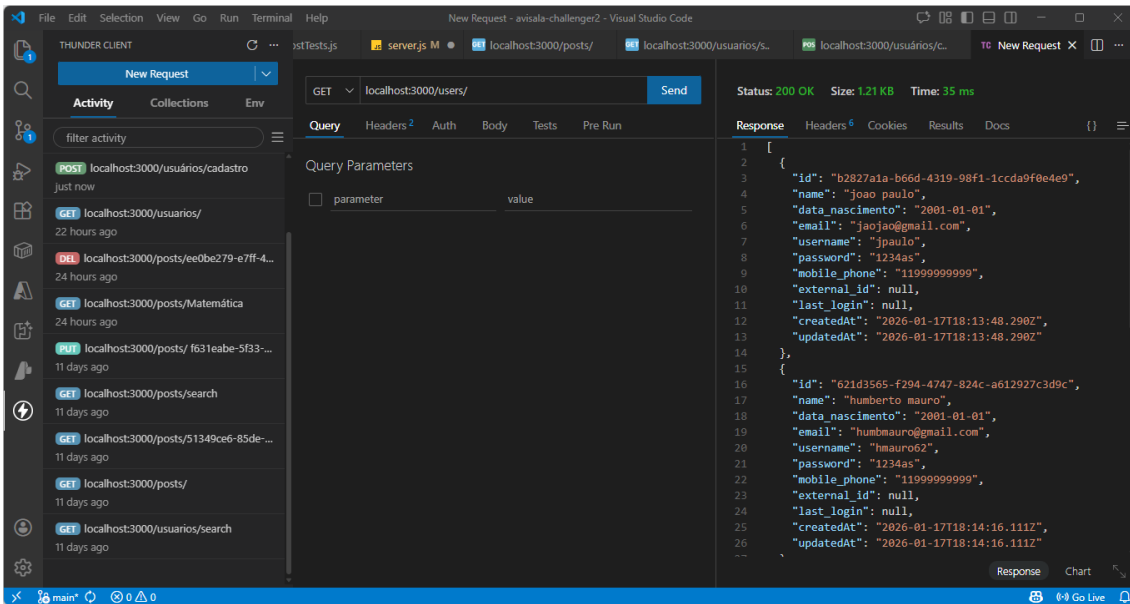
	id	name	data_nascimento	email	updated_at	created_at	username
1	b2827a1a-b66d-4319-98f1-1ccda9f0e4e9	joao paulo	2001-01-01	jaopao@gmail.com	2026-01-17 15:13:48.290 -0300	2026-01-17 15:13:48.290 -0300	jpaulo
2	621d3565-f294-4747-824c-a612927c3d9c	humberto mauro	2001-01-01	humbmauro@gmail.com	2026-01-17 15:14:16.111 -0300	2026-01-17 15:14:16.111 -0300	hmauro62
3	7758d134-0705-4928-b801-6f9fd3d2538c	Azevedo	1989-04-03	azevedo24@gmail.com	2026-01-17 21:36:30.851 -0300	2026-01-17 15:13:25.244 -0300	azev24
4	85875e79-7077-4861-972b-9be4ef59fc73	fulano	2001-01-01	fulanoo@gmail.com	2026-01-18 13:23:05.271 -0300	2026-01-18 13:23:05.271 -0300	fulano80

Atualizar Salvar Cancelar Exportar dados 200 4

4 linha(s) recuperada(s) - 0.004s (0.001s recuperado), em 2026-01-18 às 13:24:42

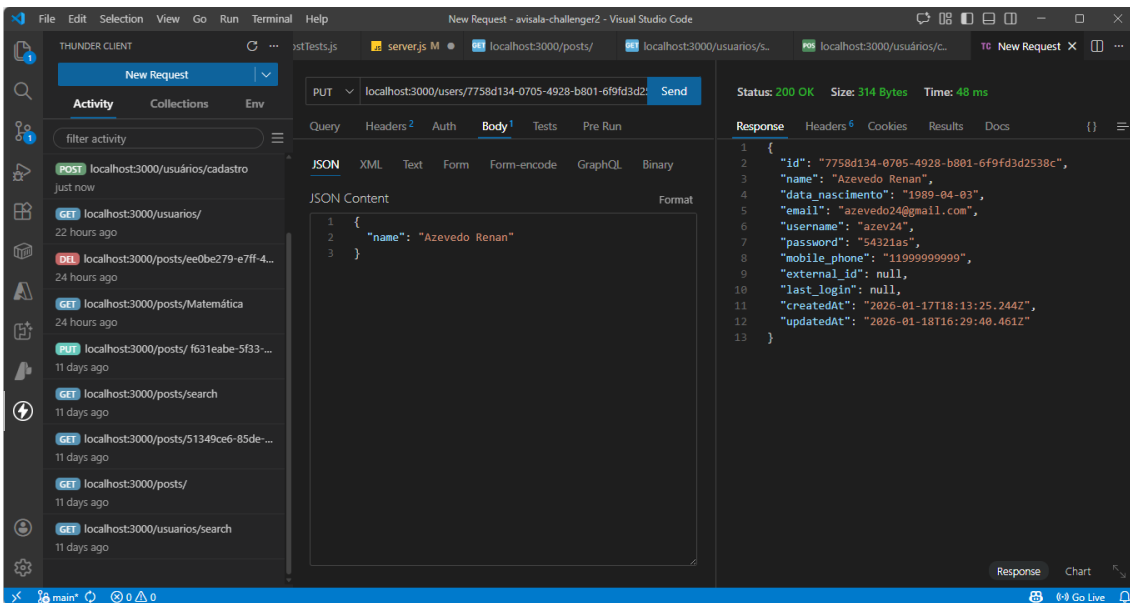
postgres avisdadb public users BRT pt_BR

GET	/	getAllUsers
-----	---	-------------



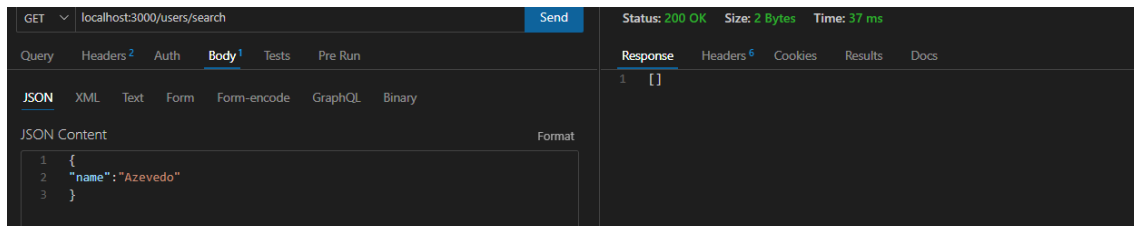
PUT	/:id	updateUser
-----	------	------------

localhost:3000/users/7758d134-0705-4928-b801-6f9fd3d2538c

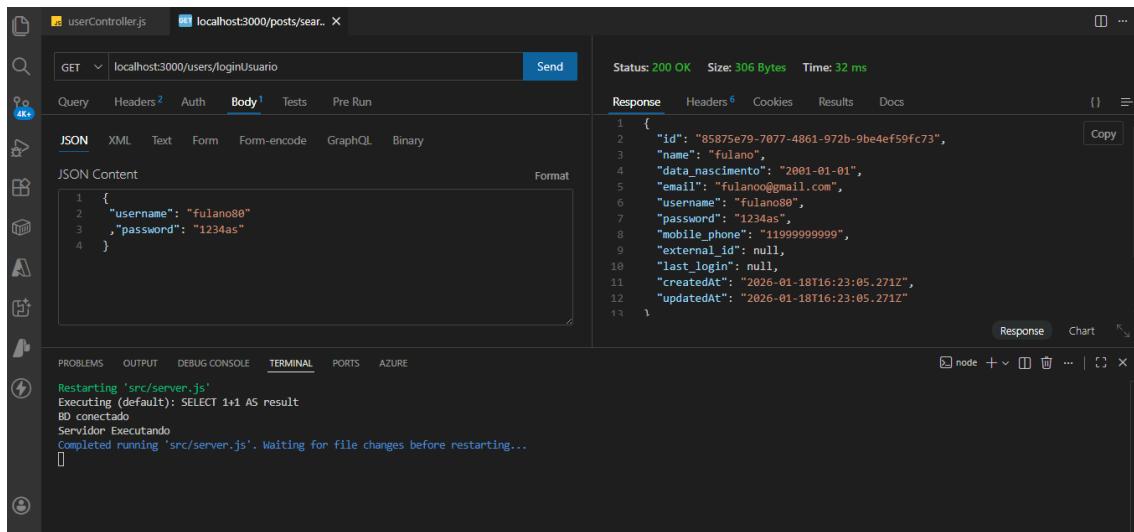


Propriedades						
Dados						
Diagrama						
Show SQL						
Insira uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)						
Grade		id	name	data_nascimento	email	updated_at
1		b2827a1a-b66d-4319-98f1-1ccda9f0e4e9	joao paulo	2001-01-01	jaojao@gmail.com	2026-01-17 15:13:48.290 -0300
2		621d3565-f294-4747-824c-a612927c3d9c	humberto mauro	2001-01-01	humbmauro@gmail.com	2026-01-17 15:14:16.111 -0300
3		85875e79-7077-4861-972b-9be4ef59fc73	fulano	2001-01-01	fulanoo@gmail.com	2026-01-18 13:23:05.271 -0300
4		7758d134-0705-4928-b801-6f9fd3d2538c	Azevedo Renan	1989-04-03	azevedo24@gmail.com	2026-01-18 13:29:40.461 -0300

GET	/search	searchByPalavraChave
-----	---------	----------------------



GET	/loginUsuario	loginUsuario
-----	---------------	--------------



Relatos de experiências e desafios enfrentados:

João Paulo:

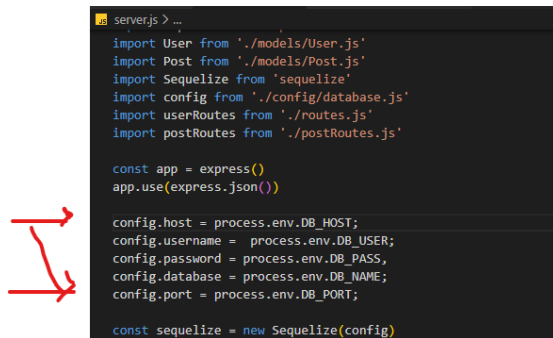
Quando executou o comando “docker compose up –build” o container do app enxergava o container do postgres, porém haviam erros na execução.

Após buscas no google, foi identificado que a versão do postgres deveria ser a 16 e não a 18. (identificado pelo Azevedo)

no docker-compose.yml: alterada a linha da imagem:

image: postgres:16-alpine # Usar uma imagem oficial e leve do Postgres

Foi necessário também, criar no server.js as instruções de config e subir novamente para o container:



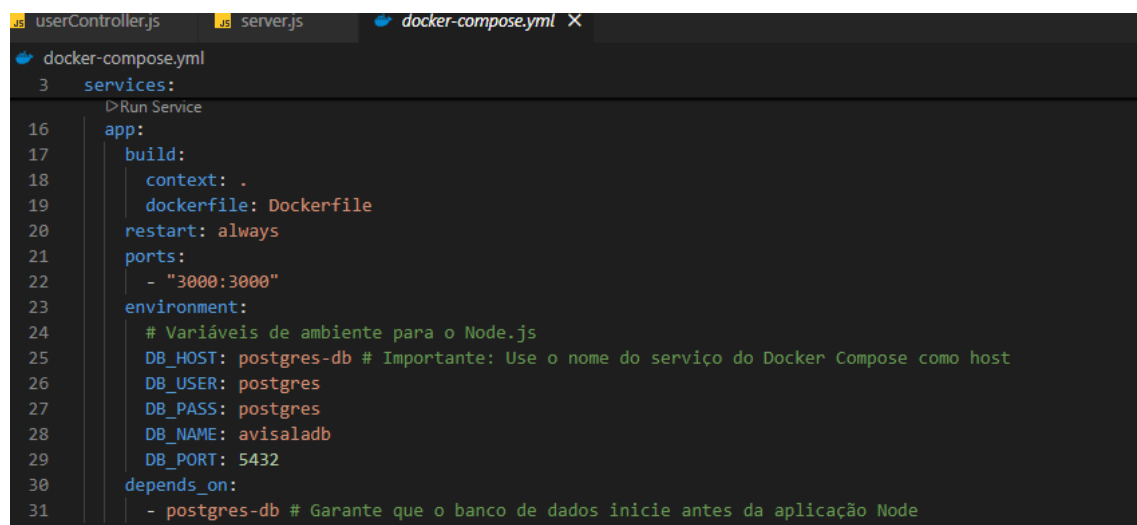
```
server.js > ...
import User from './models/User.js'
import Post from './models/Post.js'
import Sequelize from 'sequelize'
import config from './config/database.js'
import userRoutes from './routes.js'
import postRoutes from './postRoutes.js'

const app = express()
app.use(express.json())

config.host = process.env.DB_HOST;
config.username = process.env.DB_USER;
config.password = process.env.DB_PASS;
config.database = process.env.DB_NAME;
config.port = process.env.DB_PORT;

const sequelize = new Sequelize(config)
```

E no docker-compose.yml



```
docker-compose.yml
3 services:
  Run Service
16 app:
17   build:
18     context: .
19     dockerfile: Dockerfile
20   restart: always
21   ports:
22     - "3000:3000"
23   environment:
24     # Variáveis de ambiente para o Node.js
25     DB_HOST: postgres-db # Importante: Use o nome do serviço do Docker Compose como host
26     DB_USER: postgres
27     DB_PASS: postgres
28     DB_NAME: avisaladb
29     DB_PORT: 5432
30   depends_on:
31     - postgres-db # Garante que o banco de dados inicie antes da aplicação Node
```

Azevedo: criou o workflow, o Dockerfile e docker-compose, pesquisando no google e nos vídeos do curso.

Os erros encontrados foram trabalhados em conjunto com o João Paulo. O primeiro a criar a imagem no github.

Humberto: Remontei o api em minha máquina, clonando o git do João e transferir para o meu.

A primeira tentativa de subir o container, deu erro:

ERROR: failed to build: **invalid tag "***/avisala_backend-node-app:latest": invalid reference format**

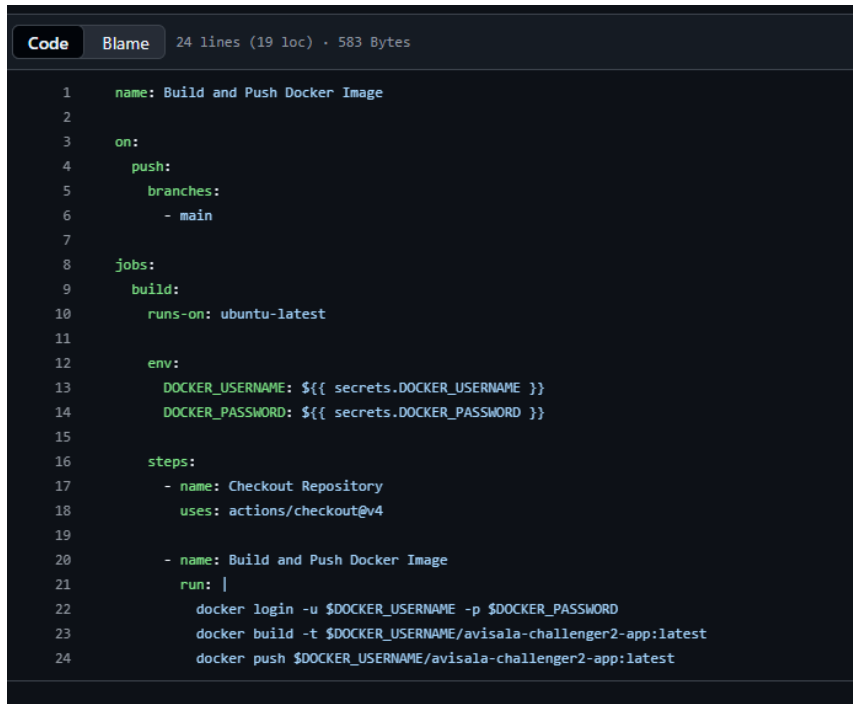
Como o dockerfile não tinha erro algum, resolvi alterar a tag da imagem para “avisala-challenger2”, por ser mais curta.

docker tag 63dcac607943 'avisala-challenger2'

e deletei as anteriores, com o container *stopado*.

`docker rmi avisala-challenger2-node-app:latest`

Altereí o workflow:



```
1  name: Build and Push Docker Image
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11
12     env:
13       DOCKER_USERNAME: ${ secrets.DOCKER_USERNAME }
14       DOCKER_PASSWORD: ${ secrets.DOCKER_PASSWORD }
15
16     steps:
17     - name: Checkout Repository
18       uses: actions/checkout@v4
19
20     - name: Build and Push Docker Image
21       run: |
22         docker login -u $DOCKER_USERNAME -p $DOCKER_PASSWORD
23         docker build -t $DOCKER_USERNAME/avisala-challenger2-app:latest
24         docker push $DOCKER_USERNAME/avisala-challenger2-app:latest
```

E refiz a subida.

Passou a dar outro erro:

ERROR: docker: 'docker buildx build' requires 1 argument

Usage: docker buildx build [OPTIONS] PATH | URL | -

Run 'docker buildx build --help' for more information

Error: Process completed with exit code 1.

A pesquisa que realizei indica que ou o Dockerfile está com erro ou o buildx está corrompido.

O buildx faz parte do Docker. Então desinstalei o Docker desktop, baixei um novo e instalei novamente. Loguei e refiz o processo. Mesmo erro.

Cheguei a alterar a versão do nodes pois da 20 a 22 há muitas melhorias, mas também não deu certo.


```
Dockerfile > ...
1 FROM node:22-alpine
2
3 WORKDIR /usr/src/app
4
5 COPY package*.json ./
6
7 RUN npm install
8
9 COPY . .
10
11 EXPOSE 3000
12
13 CMD ["node", "src/server.js"]
```

Não dei continuidade. Não consegui fazer o CI e nem o CD.

Essas são minhas Actions:

GitHub - Workflow runs - HMauro62/avisala-challenger2

Actions New workflow

All workflows Filter workflow runs

Showing runs from all workflows

12 workflow runs

	Event	Status	Branch	Actor
 uninstall jest	Build and Push Docker Image #12: Commit a657892 pushed by HMauro62	failed	main	Today at 5:17 PM 13s
 testes post e user	Build and Push Docker Image #11: Commit c7a6dc pushed by HMauro62	failed	main	Jan 17, 9:45 PM GMT-3 11s
 jest	Build and Push Docker Image #10: Commit ca1f8dc pushed by HMauro62	failed	main	Jan 17, 8:02 PM GMT-3 8s
 dockerfile node:22-alpine	Build and Push Docker Image #9: Commit 89c118b pushed by HMauro62	failed	main	Jan 17, 4:59 PM GMT-3 9s
 alterando from dockerfile para node22	Build and Push Docker Image #8: Commit 847a3d3 pushed by HMauro62	failed	main	Jan 17, 4:55 PM GMT-3 8s
 remoção dotenv	Build and Push Docker Image #7: Commit a65b4d4 pushed by HMauro62	failed	main	Jan 17, 4:34 PM GMT-3 9s
 Merge branch 'main' of github.com:HMauro62/avisala-challenger2	Build and Push Docker Image #6: Commit 81345d3 pushed by HMauro62	failed	main	Jan 17, 4:12 PM GMT-3 13s
 Rename Docker image to avisala-challenger2-app	Build and Push Docker Image #5: Commit 9bdc702 pushed by HMauro62	failed	main	Jan 17, 4:11 PM GMT-3 8s
 Fix Docker build command syntax in workflow	Build and Push Docker Image #4: Commit a0e0efc pushed by HMauro62	failed	main	Jan 17, 4:02 PM GMT-3 9s
 Update Docker image name in GitHub Actions workflow	Build and Push Docker Image #3: Commit a1097d8 pushed by HMauro62	failed	main	Jan 17, 3:50 PM GMT-3 7s
 Update Docker image name in workflow	Build and Push Docker Image #2: Commit 7a2db77 pushed by HMauro62	failed	main	Jan 17, 3:39 PM GMT-3 9s
 update from joao paulo	Build and Push Docker Image #1: Commit 52bb199 pushed by HMauro62	failed	main	Jan 17, 1:07 PM GMT-3 10s

No meu Docker desktop

Containers [Give feedback](#)

Container CPU usage  0.01% / 800% (8 CPUs available)

Container memory usage  61.91MB / 7.51GB [Show charts](#)

 ☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Actions
☐	postgres	5bade9afe6f	postgres:14	5432-5432	0%	   
☐	mongo_container_1	e94536441648	mongo	27017-27017	0%	   
☐	confident_sanderson	93a1de3b97c4	node:18		0%	   
☐	elastic_clarke	42880c04fd71	ubuntu		0%	   
☒	avisala-challenger2	-	-	-	0.01%	   
☐	app-1	5de25820c6b8	avisala-cha:3000-3000 		0%	   
☐	postgres-db-1	bf59b6245c8e	postgres:14	5432-5432 	0.01%	   

O João conseguiu. Como ele atualiza o github, não deveria dar problema.

Testes Unitário automatizado:

Users:

userController.js

```
src > controllers > JS userController.js > loginUsuario
1  import User from "../models/User.js"
2  import crypto from 'node:crypto'
3  import { Sequelize, DataTypes, Op } from 'sequelize'
4
5  export const createUser = async (req, res) => {
6    try {
7      const userToCreate = {
8        id: crypto.randomUUID(),
9        ...req.body
10      }
11
12      const user = await User.create(userToCreate)
13      res.status(201).json(user)
14    } catch(err) {
15      res.status(500).json(err)
16    }
17  }
18
19  export const getAllUsers = async (req, res) => {
20    const users = await User.findAll()
21    res.status(200).json(users)
22  }
23
24  export const deleteUser = async (req, res) => {
25    const user = await User.destroy({
26      where: {id: req.params.id}
27    })
28    res.status(200).json(user)
29  }
30
31  export const getUserByID = async (req, res) => {
32    if (req.params.id === 'search') {
33      searchByPalavraChave(req, res);
34    } if (req.params.id === 'loginUsuario') {
35      loginUsuario(req, res);
36    } else {
37      const user = await User.findByPk(req.params.id);
38      res.status(200).json(user);
39    }
40  }
```

```

41
42 export const updateUser = async (req, res) => {
43   const [affectedRows] = await User.update(
44     { name: req.body.name,
45       ...req.body },
46     {
47       where: {
48         id: req.params.id,
49       },
50     }
51   );
52   const user = await User.findByPk(req.params.id)
53   res.status(200).json(user)
54 }
55
56 export const searchByPalavraChave = async (req, res) => {
57   const users = await User.findAll({
58     where: { name: {
59       [Op.like]: "%" + req.body.text + "%"
60     } }
61   });
62   res.status(200).json(users)
63 }

```

```

64
65 export const loginUsuario = async (req, res) => {
66   const users = await User.findAll({
67     where: { username: req.body.username, password: req.body.password }
68   });
69   if (users.length === 1)
70     res.status(200).json(users[0])
71   else
72     res.status(500).json("Usuário não encontrado !");
73 }
74

```

userTests.js

```
userTests.js X
src > tests > userTests.js > ...
1  import axios from 'axios';
2
3  const URL_API = 'http://0.0.0.0:3000';
4
5  let erros = 0;
6  let acertos = 0;
7
8  async function executarTestes() {
9
10     const teste1 = 'Testando requisicao listar tudo: '
11     let userTest = '';
12     try {
13         const resposta1 = await axios.get(URL_API + '/users');
14         userTest = resposta1.data[0].id;
15         acertos = acertos + 1;
16         console.log(teste1 + 'sucesso.')
17     } catch (erro) {
18         console.error(teste1 + 'erro.');
```

```
        console.error(erro.message);
        erros = erros + 1;
    }
}

const teste2 = 'Testando requisicao listar usuario: '
try {
    const resposta2 = await axios.get(URL_API + '/users/' + userTest);
    acertos = acertos + 1;
    console.log(teste2 + 'sucesso.')
} catch (erro) {
    console.error(teste2 + 'erro.');
```

```

1      erros = erros + 1;
2    }
3
4    const teste4 = 'Testando requisicao atualizar usuario: '
5    let testeUserID = '7758d134-0705-4928-b801-6f9fd3d2538c';
6
7    try {
8      const userDataUpdate = {
9        name: 'Azevedo',
10       data_nascimento: '04.03.1989',
11       email: 'azevedo24@gmail.com',
12       username: 'azev24',
13       password: '54321as'
14     }
15
16     const resposta4 = await axios.put(URL_API + '/users/' + testeUserID, userDataUpdate);
17     acertos = acertos + 1;
18     console.log(teste4 + 'sucesso.')
19   } catch (erro) {
20     console.error(teste4 + 'erro.');
```

```

userTests.js X
src > tests > userTests.js > ...
8  async function executarTestes() {
75  const teste5 = 'Testando requisicao deletar usuario: '
76  try {
77    const resposta5 = await axios.delete(URL_API + '/users/' + testeUsuarioID);
78    acertos = acertos + 1;
79    console.log(teste5 + 'sucesso.')
80  } catch (erro) {
81    console.error(teste5 + 'erro.');
```

```

const teste7 = 'Testando login com usuario inexistente: '
try {
  const dadosLogin2 = {
    username: 'joao2',
    password: 'joao'
  }
  const resposta7 = await axios.get(URL_API + '/users/loginUsuario', {data: dadosLogin2});
  acertos = acertos + 1;
  console.log(teste7 + 'sucesso.')
} catch (erro) {
  console.error(teste7 + 'erro. ');
  console.error(erro);
  erros = erros + 1;
}

console.log('Sucesso: ', acertos)
console.log('Erros:', erros)
}

executarTestes();|

```

C:\Users\Latitude 3490\OneDrive\Documentos\A FIAP\A FASE 2\Avisala\avisala-challenger2> node src/tests/userTests.js

Testando requisicao listar tudo: sucesso.

Testando requisicao listar usuario: sucesso.

Testando requisicao inserir usuario: sucesso.

Testando requisicao atualizar usuario: sucesso.

Testando requisicao deletar usuario: sucesso.

Testando login com usuario existente: sucesso.

postController.js

```
postController.js X
src > controllers > postController.js > createPost > post
1 import Post from "../models/Post.js"
2 import crypto from 'node:crypto'
3 import { Sequelize, DataTypes, Op } from 'sequelize'
4
5 export const createPost = async (req, res) => {
6   try {
7     const postToCreate = {
8       id: crypto.randomUUID(),
9       ...req.body
10    }
11
12    const post = await Post.create(postToCreate)
13    res.status(201).json(post)
14  } catch(err) {
15    res.status(500).json(err)
16  }
17 }
18
19 export const getAllPosts = async (req, res) => {
20   const posts = await Post.findAll()
21   res.status(200).json(posts)
22 }
23
24 export const deletePost = async (req, res) => {
25   const post = await Post.destroy({
26     where: {id: req.params.id}
27   })
28   res.status(200).json(post)
29 }
30
31 export const getPostByID = async (req, res) => {
32   if (req.params.id === 'search') {
33     searchDescriptionByKeyword(req, res)
34   } else {
35     const post = await Post.findByPk(req.params.id)
36     res.status(200).json(post)
37   }
38 }
```

```

40 export const updatePost = async (req, res) => {
41   try{
42     const [affectedRows] = await Post.update(
43       { category: req.body.category,
44         ...req.body },
45       {
46         where: {
47           id: req.params.id,
48         },
49       }
50     );
51     const post = await Post.findByIdPk(req.params.id)
52     res.status(200).json(post)
53   }
54   catch (err) {
55     res.status(500).json(err)
56   }
57 }

```

```

58
59 export const searchDescriptionByKeyword = async (req, res) => {
60   const posts = await Post.findAll({
61     where: {
62       [Op.or]: {
63         description: {[Op.like]: "%" + req.body.description + "%"},
64         topic: {[Op.like]: "%" + req.body.topic + "%"}
65       }
66     }
67   });
68   res.status(200).json(posts)
69 }
70
71 export const searchByCategory = async (req, res) => {
72   try{
73     const posts = await Post.findAll({
74       where: {
75         category: {
76           [Op.eq]: req.params.category
77         }
78       }
79     });
80     res.status(200).json(posts);
81   }
82   catch (err) {
83     res.status(500).json(err)
84   }
85 }
86 }

```

postTests.js

JS postTests.js X

src > tests > JS postTests.js > executarTestesPost

```
1  import axios from 'axios';
2
3  const URL_API = 'http://0.0.0.0:3000';
4
5  let erros = 0;
6  let acertos = 0;
7
8  async function executarTestesPost() {
9
10     const teste1 = 'Testando requisicao listar todos os posts: '
11     let postTest = '';
12     try {
13         const resposta1 = await axios.get(URL_API + '/posts');
14         postTest = resposta1.data[0].id;
15         acertos = acertos + 1;
16         console.log(teste1 + 'sucesso.')
17     } catch (erro) {
18         console.error(teste1 + 'erro.');
```

```
19         console.error(erro.message);
20         erros = erros + 1;
21     }
22 }
```

```
23     const teste2 = 'Testando requisicao listar post: '
24     try {
25         const resposta2 = await axios.get(URL_API + '/posts/' + postTest);
26         acertos = acertos + 1;
27         console.log(teste2 + 'sucesso.')
28     } catch (erro) {
29         console.error(teste2 + 'erro.');
```

```
30         console.error(erro.message);
31         erros = erros + 1;
32     }
33 }
```

```
34     const teste3 = 'Testando requisicao criar post '
35     let testePostID = ''
36     try {
37         const postData = {
38             category: 'História',
39             topic: 'como destruir um País com apenas 1',
40             description: 'Donec tempus tempus justo, non commodo sapien tempus vitae. Duis scelerisque purus',
41             user_id: '7758d134-0705-4928-b801-6f9fd3d2538c'
42         }
43         const resposta3 = await axios.post(URL_API + '/posts/', postData);
44         acertos = acertos + 1;
45         console.log(teste3 + 'sucesso.')
46         testePostID = resposta3.data.id;
47     }
48     } catch (erro) {
49         console.error(teste3 + ': erro.');
```

```
50         console.error(erro.message);
51         erros = erros + 1;
52     }
53 }
```

```

54     const teste4 = 'Testando requisicao atualizar post: '
55
56     try {
57         const postDataUpdate = {
58             category: 'História',
59             topic: 'Como tornar um País Forte',
60             description: 'Donec tempus tempus justo, non commodo sapien tempus vitae. Duis scelerisque purus at
61             userid: '7758d134-0705-4928-b801-6f9fd3d2538c'
62         }
63
64         const resposta4 = await axios.put(URL_API + '/posts/' + testePostID, postDataUpdate);
65         acertos = acertos + 1;
66         console.log(teste4 + 'sucesso.')
67     } catch (erro) {
68         console.error(teste4 + 'erro.');
```

```

73     const teste6 = 'Testando requisicao buscar post por categoria: '
74     try {
75         const dadosCategoria = {
76             category: 'História'
77         };
78         const resposta6 = await axios.get(URL_API + '/posts/', {data: dadosCategoria});
79         acertos = acertos + 1;
80         console.log(teste6 + 'sucesso.')
81     } catch (erro) {
82         console.error(teste6 + 'erro.');
```

```

87     const teste5 = 'Testando requisicao deletar post: '
88     try {
89         const resposta5 = await axios.delete(URL_API + '/posts/' + testePostID);
90         acertos = acertos + 1;
91         console.log(teste5 + 'sucesso.')
92     } catch (erro) {
93         console.error(teste5 + 'erro.');
```

PS C:\Users\Latitude 3490\OneDrive\Documentos\A FIAP\A FASE
2\Avisala\avisala-challenger2> node src/tests/postTests.js

Testando requisicao listar todos os posts: sucesso.

Testando requisicao listar post: sucesso.

Testando requisicao criar post sucesso.

Testando requisicao atualizar post: sucesso.

Testando requisicao buscar post por categoria: sucesso.

Testando requisicao deletar post: sucesso.

Sucesso: 6

Erros: 0

Observação:

Caso receba o código fonte dos arquivos da estrutura um pouco diferentes é porque é muito difícil atualizar a todos. Esses *printes* de tela estão locais a minha máquina (Humberto).